PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Curso de Engenharia de Software

Análise de Eficácia de Testes com Teste de Mutação

Disciplina: Testes de Software

Nome completo: Filipe Faria Melo

Matrícula: 730453

1. Análise Inicial

1.1 Cobertura de Código (Jest)

Executou-se a suíte de testes original com `--coverage`. Todos os 50 testes passaram. A cobertura de funções atingiu 100% e a cobertura de linhas ficou em 98,64%, com apenas a linha 112 de `operacoes.js` sem cobertura (ramo de array par em `medianaArray`).

Métrica	Resultado
Statements	85,41%
Branches	58,82%
Functions	100,00%
Lines	98,64%
Linha descoberta	112

1.2 Pontuação de Mutação (Stryker)

Após instalar e configurar o StrykerJS, obteve-se na primeira execução:

Métrica	Valor
Mutation Score	73,71%
Mutantes gerados	213
Killed	154
Survived	**56**
Timeout	3
No coverage	0

1.3 Discrepância Entre as Métricas

A diferença entre **98,64% de cobertura** e **73,71% de pontuação de mutação** é o resultado central desta análise. Cobertura responde: *"essa linha foi executada?"*. Teste de mutação vai além: introduz modificações deliberadas no código e verifica se algum teste falha. Se nenhum detecta a mudança, o mutante sobrevive, evidenciando que aquela lógica não está sendo verificada. Em síntese: cobertura prova que o código foi *executado*; pontuação de mutação prova que o comportamento foi *verificado*.

2. Análise de Mutantes Críticos

Mutante 1: medianaArray (Ramo Par Nunca Testado)

Arquivo:	src/operacoes.js, linha 112
Tipo:	ArithmeticOperator

```
// Original: return (sorted[mid - 1] + sorted[mid]) / 2; // Mutação: + trocado por - return
(sorted[mid - 1] - sorted[mid]) / 2;
```

Por que sobreviveu: O único teste usava `[1,2,3,4,5]` (ímpar), logo o bloco `if (length % 2 === 0)` nunca era ativado. Qualquer mutação na linha 112 passava invisível.

Mutante 2: isMaiorQue / isMenorQue (Sem Caso de Fronteira)

Arquivo:	src/operacoes.js, linhas 104-105
Tipo:	RelationalOperator

```
// Original: function isMaiorQue(a, b) { return a > b; } // Mutacao: function isMaiorQue(a, b) {
return a >= b; }
```

Por que sobreviveu: Os testes só verificavam o caminho `true` com valores claramente distintos (`isMaiorQue(10,5)`). Para esses valores, `a > b` e `a >= b` produzem o mesmo resultado. O caso de fronteira (valores iguais) estava ausente.

Mutante 3: clamp (Mutante Equivalente)

Arquivo:	src/operacoes.js, linhas 88-90
Tipo:	RelationalOperator

```
// Original: if (valor < min) / if (valor > max) // Mutacao: if (valor <= min) / if (valor >= max)
```

Por que sobreviveu: Único teste: `clamp(5,0,10)` com valor estritamente dentro do intervalo, sem ativar nenhum `if`. Mutante equivalente: quando `valor === min`, retornar `min` ou `valor` produz o mesmo resultado.

3. Solução Implementada

3.1 medianaArray com Array Par

```
test('deve calcular a mediana de um array par', () => { expect(medianaArray([1, 2, 3, 4])).toBe(2.5);
});
```

Força o ramo `if (length % 2 === 0)` a executar. A mutação retornaria `-0.5` em vez de `2.5`, matando o mutante.

3.2 Fronteiras para isMaiorQue e isMenorQue

```
expect(isMaiorQue(5, 5)).toBe(false); // mata a >= b expect(isMenorQue(5, 5)).toBe(false); // mata a
<= b
```

Com `a === b`, o mutante `a >= b` retorna `true` enquanto o original retorna `false`. Diferença detectada.

3.3 Mensagens de Erro (StringLiteral)

O Stryker substituiu cada mensagem de erro por string vazia. Testes de exceção foram atualizados para verificar a mensagem exata:

```
// Antes: expect(() => divisao(5,0)).toThrow(); // Depois: expect(() => divisao(5,0)).toThrow('Divisao
por zero nao e permitida.');
```

3.4 Demais Mutantes Sobreviventes

Função	Novos casos adicionados
isPrimo	isPrimo(1)=false, isPrimo(2)=true, isPrimo(4)=false
celsiusParaFahrenheit	100°C=212°F (evita mascaramento por valor zero)
fatorial	fatorial(0)=1, fatorial(1)=1
fibonacci	fibonacci(0)=0, fibonacci(1)=1
mdc / mmc	mdc(12,8)=4 e mmc(4,6)=12
Conversões angulares	grausParaRadianos(90) e radianosParaGraus(PI/2)

4. Resultados Finais

Métrica	Antes	Depois
Mutation Score	73,71%	96,71%
Killed	154	203
Survived	**56**	**7**
Timeout	3	3
Total mutantes	213	213
Testes na suíte	50	120

A pontuação passou de 73,71% para **96,71%**, com 49 mutantes a mais mortos. Os 7 restantes são mutantes equivalentes.

4.1 Mutantes Equivalentes

Localização	Por que é equivalente
fatorial linha 19 (4x)	Loop for(i=2;i<=n) nunca executa para n=0 ou n=1; resultado permanece 1 com ou sem early return
produtoArray linha 84	[].reduce(...,1) com array vazio já retorna 1 (valor inicial do reduce)
clamp linhas 88-89	Quando valor===min, retornar min ou valor tem o mesmo efeito; idem para max

Score efetivo excluindo equivalentes: (203+3)/(213-7) = 206/206 = **100%**. Toda lógica distinguível por testes está coberta.

5. Conclusão

Este trabalho demonstrou a diferença fundamental entre cobertura de código e eficácia de testes. A suíte original atingia 98,64% de cobertura, mas a pontuação de mutação de 73,71% revelou que 56 bugs artificiais passariam despercebidos.

#	Categoria de fraqueza	Exemplo
1	Caminhos não percorridos	Ramo de array par em medianaArray nunca ativado
2	Ausência de casos de fronteira	isMaiorQue(5,5) nunca testado
3	Valor zero mascarando mutações	celsius=0 oculta erros aritméticos

A adição de testes direcionados elevou a pontuação para **96,71%** com 120 testes. Os 7 mutantes equivalentes restantes são impossíveis de matar; excluindo-os, o score efetivo é 100%. O teste de mutação é hoje a métrica mais rigorosa para avaliar uma suíte de testes.